

Class #12

04.02 Data Movement and Resilience Patterns

Software Architectures · Master in Informatics Engineering
Data ownership, reliable events, duplicate handling, and
dependency failure

Agenda

- Block 1 - ADR 12.1: data ownership and table boundaries 45 min
- Exercise - evaluation forces matrix for ADR 12.1 20 min
- Block 2 - ADR 12.2: transactional outbox 35 min
- Block 3 - ADR 12.3: inbox / idempotent consumer 30 min
- Block 4 - ADR 12.4: circuit breaker, retry, and health signals 30 min
- References - pattern catalogues references

ADR 12.1

Data ownership for ticket processing

Several services need the same data. Which service owns it, and how do the others get what they need?

Source chapter: *Software Architecture: The Hard Parts*, Chapter 9, "Data Ownership and Distributed Transactions".

Source ADRs: *Software Architecture: The Hard Parts*, Appendix B, "Single Table Ownership for Bounded Contexts" and "Survey Service Owns the Survey Table".

Pattern reference: [Microservices.io Database per Service](https://microservices.io/Database%20per%20Service).

1. Context

Sysops Squad is being decomposed into services and bounded contexts.

At the same time, the database team and development team need to assign table ownership.

The source material separates the problem into three cases:

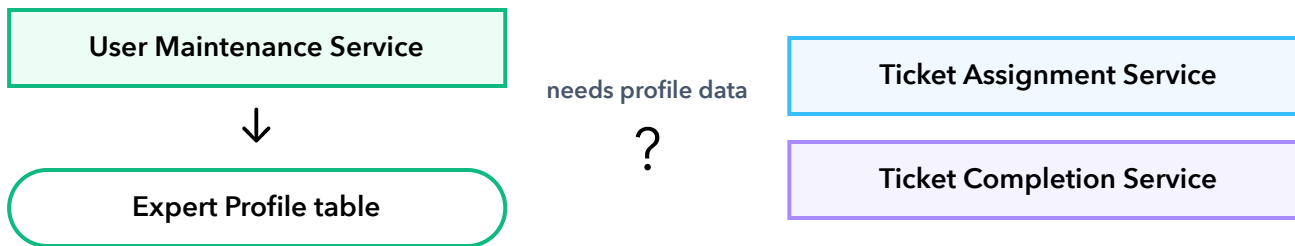
Case	What is happening
Single ownership	one service writes a table; other services may still need reads
Common ownership	most services write the same kind of cross-cutting table
Joint ownership	two services in the same domain write the same table

Today we use the single-ownership and joint-ownership Sysops cases.

1. Context

Single ownership with read pressure

The Expert Profile table is maintained by User Maintenance, but ticketing services need profile information during ticket assignment and completion.



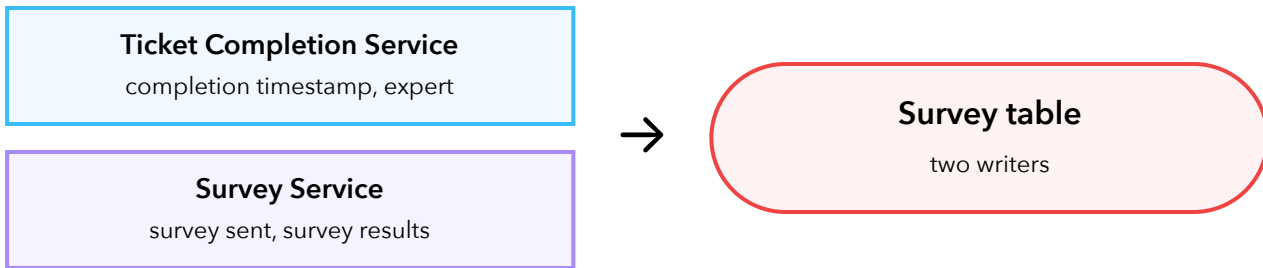
Question: what access path should the ticketing services use?

1. Context

Joint ownership on the Survey table

Ticket Completion writes completion data. Survey Service writes survey sending and survey-result data.

The source case says table splitting is not available because of the Survey table structure.



Question: how do we remove joint write ownership without putting everything back into one database boundary?

2. Proposed solutions

For the single-ownership case, compare read-access options.

Option	What it tries to preserve
Direct table access	shortest read path
Owner API	strict ownership through the owning service
Replicated read model / cache	local read performance and availability

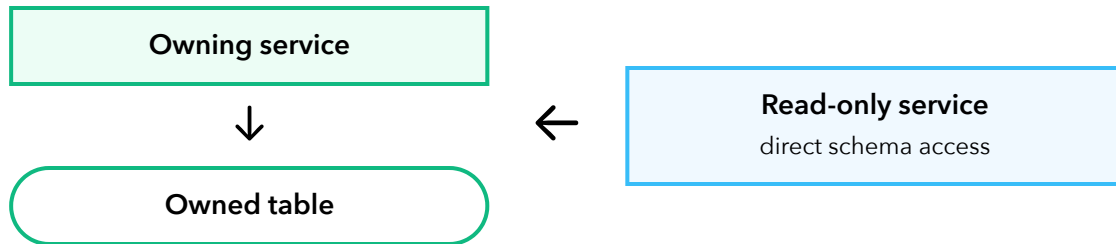
For the Survey table, compare the two source options.

Option	What it tries to preserve
Common shared data domain	both services can write the shared table
Delegation to Survey Service	one service writes; the other sends needed data

Data Ownership

How each option works

Option 1: direct table access



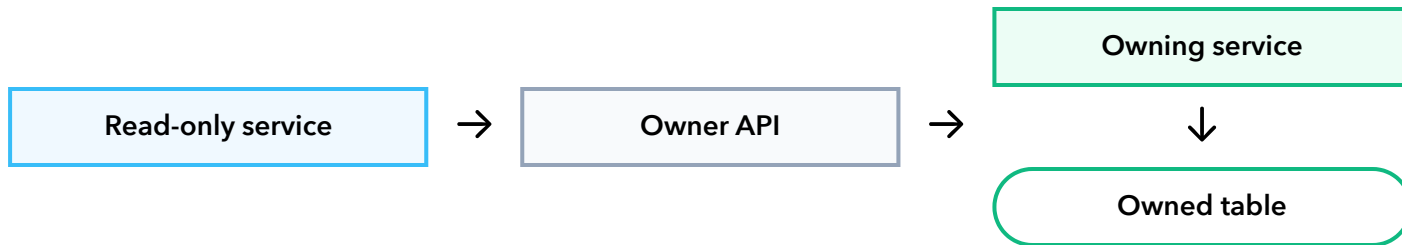
Mechanism: the read-only service queries the owner's table directly.

Works when the system is still one ownership and deployment unit.

May fail because schema changes leak across contexts and the owner cannot change its table independently.

Operational burden: every schema change needs consumer discovery because hidden reads are easy.

Option 2: owner API



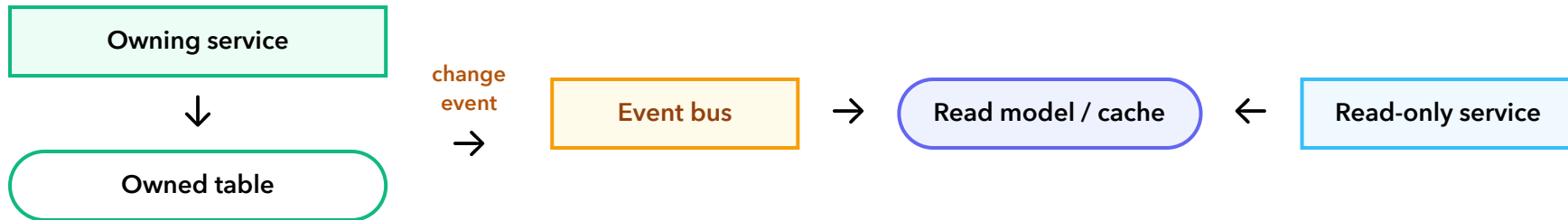
Mechanism: consumers ask the owning service for the data instead of reading its table.

Works when correctness and ownership matter more than the fastest read path.

May fail when high-volume reads create latency, availability, or coupling pressure.

Operational burden: API contracts, versioning, timeouts, and fallback behavior become part of the data access design.

Option 3: replicated read model / cache



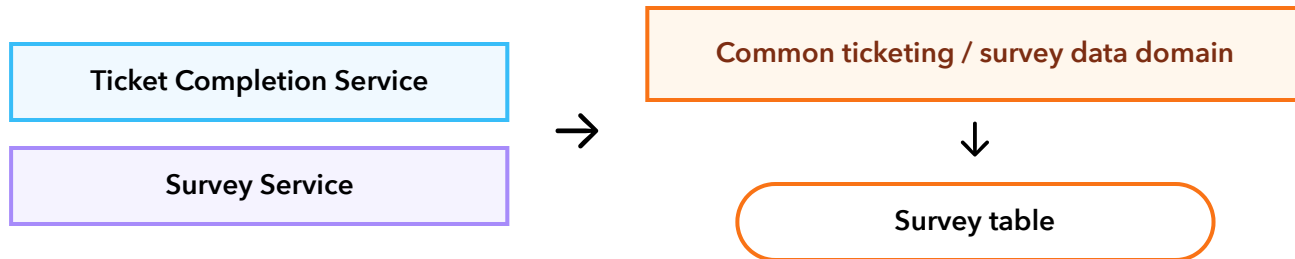
Mechanism: the owner publishes changes and consumers maintain local read data.

Works when reads must be local, fast, or resilient to owner-service outages.

May fail when stale data is treated as authoritative or repair and replay are ignored.

Operational burden: freshness, replay, repair, retention, and cache invalidation must be visible.

Option 4: common shared data domain



Mechanism: keep the Survey table in a shared data boundary both services can write.

Works when the shared boundary is honest and tightly governed.

May fail because broadening the schema can pull the decomposed system back toward a shared database.

Operational burden: coordinated schema change, shared permissions, and shared regression risk.

Option 5: delegation to Survey Service



Mechanism: Ticket Completion sends the data needed to start the survey; Survey Service inserts the Survey record.

Works when the survey process can be triggered by a message carrying the required completion data.

May fail if the event payload is incomplete, lost, duplicated, or treated as an invisible side effect.

Operational burden: event contract, reliable publication, idempotent consumption, and monitoring.

3. Technical analysis

Option	Works because	May fail because
Direct table access	shortest read path	schema and ownership coupling remain hidden
Owner API	protects the owning service boundary	latency and availability coupling
Replicated read model	gives local reads	stale data and repair complexity
Common shared data domain	keeps current write shape	shared database coupling returns
Delegation to Survey Service	creates one writer	message reliability and duplicate handling matter

Evaluation forces

For this ADR, the dominant forces are:

- data ownership
- bounded-context integrity
- data consistency
- read performance / availability
- fault isolation
- operational supportability

Accepted damage:

- latency through owner APIs
- stale read models
- shared schema governance
- reliable publication and duplicate handling

Exercise

Build the evaluation forces matrix for ADR 12.1

20 min

Compare the options against the forces before choosing.

Matrix skeleton

Option	Deployability	Consistency	Operations	Fault isolation	Ownership	Supportability	Reversibility
Direct table access	high short-term	fresh reads, unsafe boundary	low until schemas change	low	unclear	weak under change	easy now, costly later
Owner API							
Replicated read model							
Shared data domain							
Delegation							

The value is in the explanation: which forces matter, which damage you accept, and why that trade-off fits this scenario.

What's the verdict and why?

ADR 12.1

Final solution

Use explicit table ownership.

For single-table ownership:

- the only service that writes and maintains the table owns it
- read-only services in another bounded context do not access the owner database or schema directly

For the Survey table, move to one writer: Survey Service owns the Survey table.

Book ADR verdict

The book records two decisions that matter here.

Single Table Ownership for Bounded Contexts: assign the table to the only service that writes and maintains it; services outside the bounded context cannot directly access the database or schema containing that table.

Survey Service Owns the Survey Table: Survey Service becomes the only writer. Ticket Completion sends the necessary survey data when it triggers the customer survey process.

That second decision creates the handoff to outbox and inbox: the event now needs reliable publication and duplicate-safe consumption.

ADR 12.2

Use transactional outbox for reliable domain events

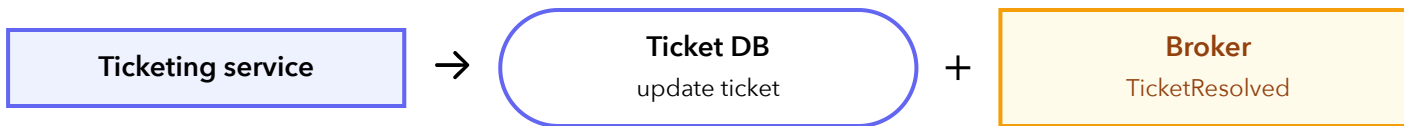
A service changes local state. Other services must learn that fact.

References: [Microservices.io Transactional Outbox](#); [Azure transactional outbox](#).

1. Context

Source context from Microservices.io: a command often updates aggregates and sends messages; two-phase commit (2PC) is often unavailable or undesirable; if the DB commits, messages must be sent, and if the DB rolls back, messages must not be sent.

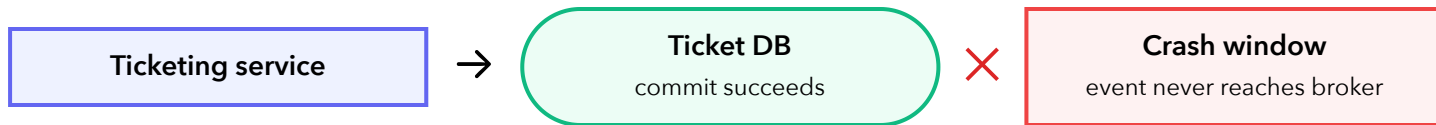
Sysops Squad example: resolving a ticket updates ticket state and should notify billing, reporting, or customer-facing status views.



The danger is the gap between commit and publish.

Source force to keep visible: the message must describe committed state, not hoped-for state.

The failure window



From a student perspective: the bug is not "events are hard"; the bug is that two systems must agree after one of them has already committed.

2. Proposed solutions

Option	What it tries to protect
Publish after commit	business state first
Publish before commit	propagation first
Distributed transaction / 2PC	atomicity across systems
Transactional outbox	atomic local write, async publish
Event sourcing	events as the stored source of truth

Reliable Publication

The failure is between two systems

Transactional outbox does not remove asynchronous complexity. It moves the risky part into a durable local record that can be retried.

Option 1: publish after commit



Mechanism: finish the local transaction, then call the broker.

Acceptable when the event is advisory and can be repaired manually.

Failure mode: state commits, service crashes, event is never published.

Operational burden: reconciliation jobs or human repair become necessary.

Option 2: publish before commit



Mechanism: publish first, then attempt the local database commit.

Acceptable only when consumers can tolerate provisional or retractable facts.

Failure mode: event is published, commit rolls back, consumers act on a false fact.

Operational burden: compensating or correction events become mandatory.

Option 3: distributed transaction / 2PC



Mechanism: coordinate database and broker through a distributed transaction.

Acceptable when the platform supports it and the coupling is worth the guarantee.

Failure mode: the slowest or least available participant becomes part of the commit path.

Operational burden: transaction coordinator, participant compatibility, and failure recovery.

Option 4: transactional outbox



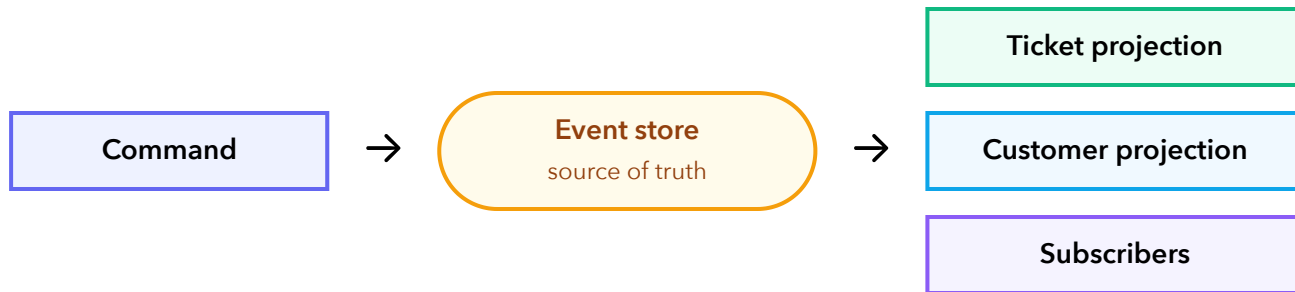
Mechanism: write business state and an outbox message in the same local transaction; a relay publishes later.

Acceptable when the event matters and 2PC is not a good option.

Failure mode: the relay can publish the same message more than once.

Operational burden: relay monitoring, ordering rules, cleanup, and idempotent consumers.

Option 5: event sourcing



Mechanism: store domain events as the source of truth and derive projections from them.

Acceptable when audit, replay, and temporal modeling are central requirements.

Failure mode: query models, schema evolution, and replay semantics become hard.

Operational burden: event versioning, projection repair, and stronger modeling discipline.

3. Technical analysis

Option	Works because	May fail because
Publish after commit	simple request path	crash loses event
Publish before commit	fast propagation	event can describe rolled-back state
2PC	direct atomicity goal	infrastructure and availability coupling
Outbox	one local transaction protects state + message record	duplicates and relay operations remain
Event sourcing	event log is authoritative	large modeling and operational shift

Evaluation forces

For ADR 12.2, the dominant forces are:

- data consistency
- fault isolation
- observability / supportability

Accepted damage: relay operations, duplicate delivery, and retention policy.

What's the verdict and why?

ADR 12.2

Final solution

Use transactional outbox for important domain events.

Write the business state and the message record in the same local transaction. Let a relay publish the message later.

Pair it explicitly with idempotent consumers.

Pattern verdict

Outbox protects publication from the local write.

It does not make delivery exactly once.

The accepted consequences are relay operations, duplicate delivery, retention policy, and monitoring.

The problem was atomicity between local state and message publication without relying on distributed transactions.

ADR 12.3

Use Inbox / Idempotent Consumer for message handling

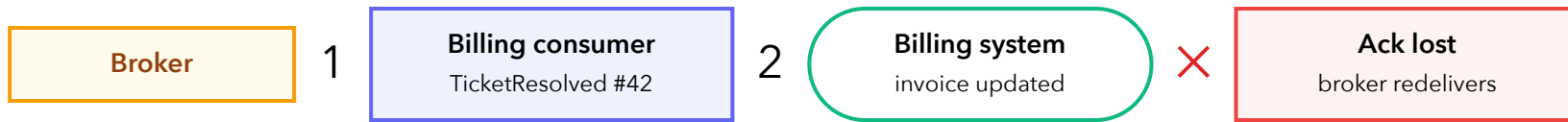
A broker can deliver the same message twice. The destination system must apply the business effect once.

Reference: [Microservices.io Idempotent Consumer](https://microservices.io Idempotent Consumer).

1. Context

Source context from [Microservices.io](https://microservices.io): at-least-once delivery means the same consumer can be invoked repeatedly for the same message; a non-idempotent handler can create incorrect business results.

Sysops Squad example: a billing consumer receives `TicketResolved #42` from the broker and updates the billing system. If acknowledgement is lost, the broker can redeliver the same event even though the invoice update already happened.



Duplicate delivery is normal. Duplicate side effects are the bug.

Why this follows outbox

Outbox makes event publication reliable.

Reliable publication often means at-least-once delivery, not exactly-once effects.

So the next design question is on the consumer side: where do we record that this message has already reached the destination system?

2. Proposed solutions

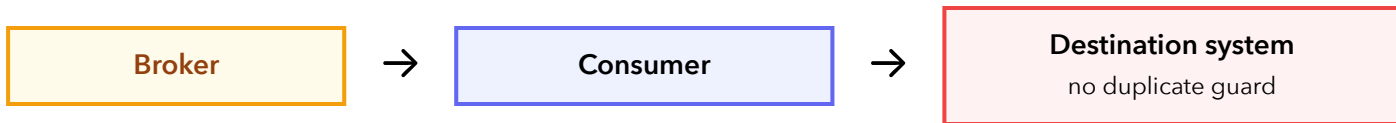
Each option decides where duplicate detection lives between broker, consumer, and destination system.

Option	Where duplicate handling lives
Trust the broker	nowhere in the consumer
Deduplicate in memory	inside the running consumer process
Check destination state	in current business state
Inbox table	durable message log beside the destination write
IDs in affected entities	inside the entity being changed

Idempotency

One message, one business effect

Option 1: trust the broker

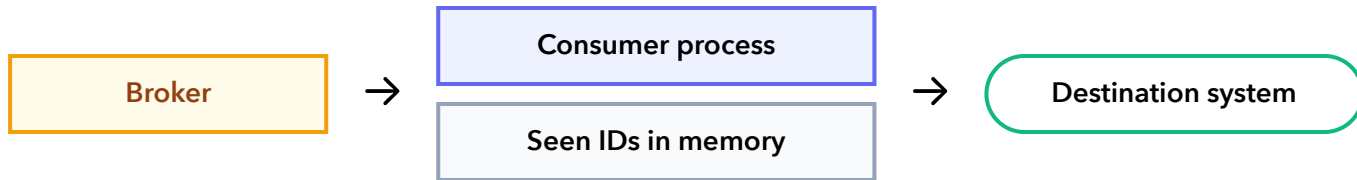


Mechanism: consume the message and apply the destination update with no duplicate protection.

Acceptable only when the handler is naturally idempotent.

May fail because retries, acknowledgements, and consumer crashes are enough to create duplicates.

Option 2: deduplicate in memory

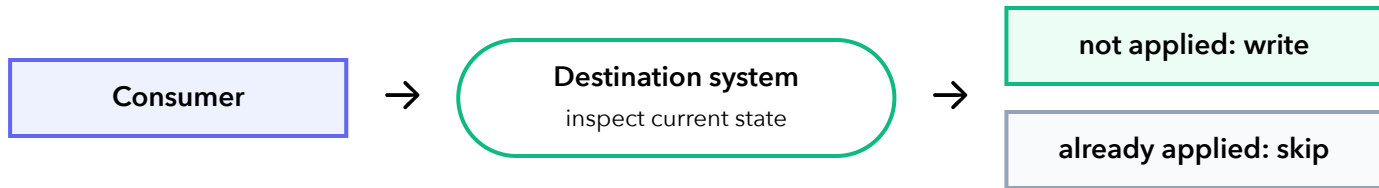


Mechanism: remember recently seen message IDs inside the running consumer process before updating the destination.

Acceptable only for local optimization, not correctness.

May fail after restart, scale-out, or message rebalancing.

Option 3: check destination state



Mechanism: inspect destination state before applying the message effect.

Acceptable when the domain state unambiguously proves prior handling.

May fail when two different messages lead to the same state or the side effect is external.

Option 4: Inbox table



Mechanism: in the consumer's local transaction, record the message ID in an Inbox table and apply the destination update.

Acceptable when duplicate processing causes real harm and the destination update can share a durable transaction with the Inbox record.

May fail if message IDs are unstable, retention is ignored, or the destination effect is outside the same transaction.

Option 5: message IDs in affected entities



Mechanism: store processed message IDs on the destination entity being changed.

Acceptable when each event affects a small, clear set of entities.

May fail for cross-entity effects, long histories, or events that trigger external calls.

3. Technical analysis

Option	Works because	May fail because
Trust broker	no local complexity	at-least-once delivery permits duplicates
Memory dedupe	cheap and fast	restart and scale-out erase memory
Destination-state check	uses business state	edge cases and external effects leak
Inbox table	durable, explicit message identity	adds storage and transaction work
IDs in entity	localizes duplicate check	awkward for broad side effects

Evaluation forces

For ADR 12.3, the dominant forces are:

- data consistency
- supportability
- fault isolation

Accepted damage: extra writes, retention policy, and a stronger event identity contract.

What's the verdict and why?

ADR 12.3

Final solution

Use the Inbox / Idempotent Consumer pattern where repeated handling causes harm.

The consumer records message identity durably while applying the destination update. For simple entity-bound effects, the processed message ID can live on the destination entity itself.

The consumer must make repeated delivery safe.

Pattern verdict

Inbox / Idempotent Consumer solves duplicate side effects between broker, consumer, and destination system.

It does not solve event ordering, stale reads, or wrong business compensation.

The accepted consequences are extra writes, retention policy, and a stronger event identity contract.

Outbox makes publication reliable; Inbox / Idempotent Consumer makes repeated delivery safe.

ADR 12.4

Circuit breaker and health around a remote dependency

What should a service do when a dependency is slow, failing, or only half alive?

References: Azure Circuit Breaker; Azure Health Endpoint Monitoring.

1. Context

Azure's circuit breaker pattern frames the problem as remote faults that can take different amounts of time to recover from.

Sysops Squad-style example: ticket creation calls a remote entitlement, payment, or customer-status verifier before accepting a support request.



Repeated calls during an outage can turn one dependency failure into system pressure.

Health endpoint monitoring is a signal for callers or operators. It is not proof that the whole business workflow is healthy.

2. Proposed solutions

Option	What it tries to protect
Timeout only	request threads
Retry only	transient failure recovery
Retry with backoff	dependency pressure
Circuit breaker	caller capacity and recovery time
Async verification	business workflow isolation
Health endpoint / heartbeat	operational visibility and routing signals

Dependency Failure

Calls, retries, breakers, and signals

Timeout only

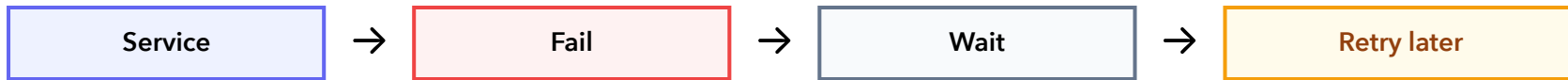


Mechanism: bound how long the caller waits.

Acceptable as the minimum protection for synchronous calls.

May fail because every request can still hit the sick dependency.

Retry with backoff

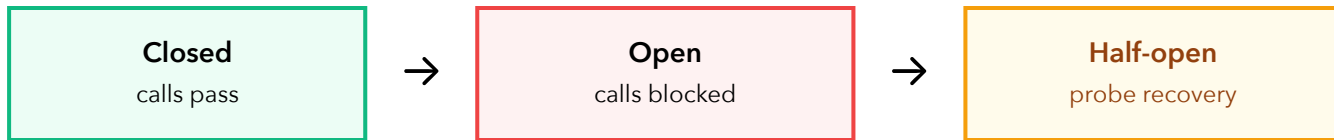


Mechanism: repeat the call after a delay, ideally with backoff.

Acceptable when failures are brief and the operation is safe to retry.

May fail because retries can amplify load and duplicate unsafe operations.

Circuit breaker



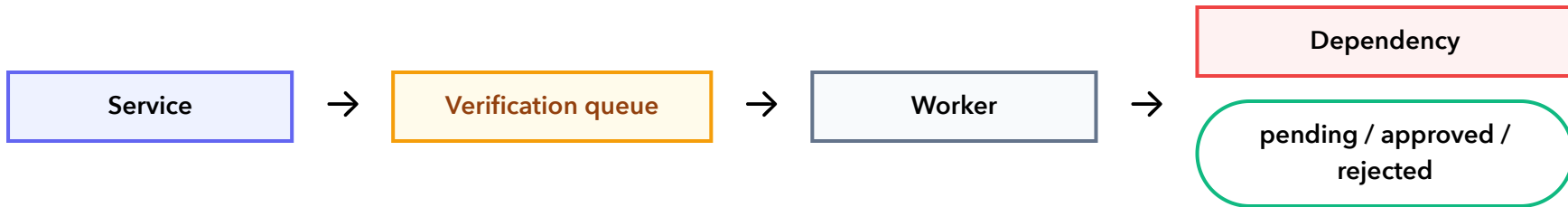
Threshold reached opens the breaker; a successful probe closes it; a failed probe opens it again.

Mechanism: move from closed to open after failure thresholds, then probe half-open recovery.

Acceptable when continuing to call makes the caller and dependency worse.

May fail if thresholds are naive or degraded behavior was never designed.

Async verification



Mechanism: queue the verification and process it outside the request path.

Acceptable when the business can expose pending state.

May fail because the business process now has pending states and delayed answers.

Health endpoint / heartbeat



Mechanism: expose liveness/readiness/dependency checks to monitors or routing components.

Acceptable when each signal has a clear consumer and meaning.

May fail when it only proves the process can return HTTP 200, or when deep checks overload the dependency being checked.

Operational burden: define who consumes the signal, how often it runs, what it checks, and what happens when it is red.

3. Technical analysis

Option	Works because	May fail because
Timeout	prevents infinite waits	still pounds dependency
Retry/backoff	handles brief faults	can amplify outages
Circuit breaker	protects caller capacity	needs tuning and fallback
Async verification	isolates request path	changes business workflow
Health endpoint	improves routing and alerting	can lie or overload checks

Evaluation forces

For ADR 12.4, the dominant forces are:

- fault isolation
- operational complexity
- supportability

Accepted damage: degraded behavior, threshold tuning, and false-open risk.

What's the verdict and why?

ADR 12.4

Final solution

Use timeout, bounded retry, and circuit breaker for synchronous dependencies.

Use health/readiness signals for operations and routing.

Define the degraded behavior before the breaker opens.

Pattern verdict

The decision protects the caller from cascading failure.

Health/readiness signals are useful operational signals, but they are not proof that the business workflow is healthy.

The accepted consequences are degraded behavior, threshold tuning, false-open risk, and operational monitoring.

The problem was cascading failure, not the absence of a pattern catalogue entry.

Pattern Catalogues

Use after the ADR question is clear

References and what they are good for

Source	Good for
Azure Architecture Center - Cloud Design Patterns	problem/solution/issues format for retry, circuit breaker, health, sidecar, saga
Microsoft patterns & practices GitHub samples	concrete sample implementations
AWS Prescriptive Guidance - Cloud design patterns	cross-checking pattern names and modernization contexts
AWS Prescriptive Guidance Patterns	more recipe-like implementation catalogue
Google Cloud Architecture Framework	reliability, security, cost, operational excellence framing
Microservices.io patterns	transactional outbox, idempotent consumer, messaging, saga, database-per-service

Catalogues help after you know which failure or trade-off the ADR is addressing.